

Git 스테이징 영역과 로컬 저장소의 차이 및 생성 위치 (Windows 기준)

Git에서 버전 관리를 수행할 때 핵심이 되는 세 가지 논리적 공간(작업 디렉터리, 스테이징 영역, 로컬 저장소) 중 스테이징 영역과 로컬 저장소의 개념 차이와 Windows 환경에서의 실제 물리적 생성 위치에 대한 안내 문서입니다.

1. 개념의 정의 및 차이점

Git은 변경 사항을 바로 저장소에 기록하지 않고 중간에 완충 지대를 두는 구조를 가지고 있습니다.

- **스테이징 영역 (Staging Area / Index):** 다음 커밋(Commit)에 포함할 변경 사항들을 임시로 저장하고 준비하는 완충 지대입니다. 작업 디렉터리에서 수정된 파일 중 `git add` 명령어를 통해 선택된 파일들만 이 영역에 반영됩니다.
- **로컬 저장소 (Local Repository):** 제출된 파일들의 버전(커밋) 히스토리가 영구적으로 안전하게 기록되는 공간입니다. `git commit` 명령어를 실행하면 스테이징 영역에 있던 변경 사항들이 하나의 버전으로 묶여 로컬 저장소에 저장됩니다.

스테이징 영역이 필요한 이유: 작업 디렉터리에서 여러 개의 파일을 동시에 수정하더라도, 논리적으로 연관된 변경 사항들만 선택적으로 묶어서(`git add`) 개별적인 커밋으로 기록할 수 있기 때문입니다. 이를 통해 커밋 히스토리를 깔끔하고 명확하게 유지할 수 있습니다.

2. Windows 기준 물리적 생성 위치

스테이징 영역과 로컬 저장소는 별도의 외부 서버나 프로그램 내부에 존재하는 것이 아니라, 사용자가 프로젝트를 진행하는 폴더 내부의 숨겨진 디렉터리 안에 파일과 폴더 형태로 생성됩니다.

특정 폴더에서 `git init` 명령어를 실행하면 해당 폴더 내부에 **.git**이라는 이름의 숨김 폴더가 자동으로 생성되며, 이 폴더 내부에서 두 영역이 다음과 같이 관리됩니다.

- **기본 경로:** C:\Users\사용자이름\Documents\프로젝트폴더\.git\

영역별 상세 매핑 구조

- **스태이징 영역의 실제 위치:** `.git/index` (파일)
 - `.git` 폴더 바로 아래에 생성되는 `index`라는 이름의 바이너리 파일입니다. 사용자가 `git add`를 실행하면 다음 커밋에 포함될 파일의 경로, 해시값 등의 명세서 정보가 이 파일 내에 기록됩니다.
- **로컬 저장소의 실제 위치:** `.git/objects/` 및 `.git/refs/` (폴더)
 - 사용자가 `git commit`을 실행하면 스태이징 영역에 있던 데이터를 바탕으로 실제 압축된 파일 객체들이 `objects` 폴더 내에 영구적으로 생성되며, 브랜치와 태그 정보는 `refs` 폴더에서 관리됩니다.

3. 주요 흐름 명령어 요약

- **작업 디렉터리 → 스태이징 영역:** `git add <파일명>`
- **스태이징 영역 → 로컬 저장소:** `git commit -m "커밋 메시지"`
- **상태 확인:** `git status` 실행 시 초록색 파일은 스태이징 영역에 있는 상태, 빨간색 파일은 작업 디렉터리에만 머물러 있는 상태를 의미합니다.

Git add 명령과 커밋의 차이점 이해하기

git add . 명령은 변경이 있는 파일들을 '커밋할 준비 상태(Staging Area)'로 만드는 것이지, 실제로 커밋을 수행하는 것은 아닙니다. Git의 작동 구조와 해당 명령의 정확한 역할을 단계별로 나누어 설명합니다.

1. Git의 3가지 공간

Git은 소스코드를 관리할 때 크게 3가지 공간을 거치게 됩니다.

- **Working Directory (작업 디렉토리):** 사용자가 실제로 파일을 수정, 생성, 삭제하는 PC의 실제 폴더 공간입니다.
- **Staging Area (스테이징 영역):** 다음 커밋에 포함할 파일들을 임시로 모아두는 준비 공간입니다.
- **Repository (저장소):** git commit을 통해 스냅샷이 영구적으로 저장되는 공간입니다.

git add .는 파일의 상태를 Working Directory에서 Staging Area로 이동시키는 역할을 합니다.

2. git add . 의 정확한 역할

git add . 명령은 현재 디렉토리내 그 하위 디렉토리 내에서 변경이 발생한 모든 파일을 Staging Area에 올립니다. 여기서 '변경'이란 다음 3가지를 모두 포함합니다.

- 새롭게 생성된 파일 (Untracked files)
- 기존에 존재하던 파일 중 내용이 수정된 파일 (Modified files)
- 기존에 존재하던 파일 중 삭제된 파일 (Deleted files)

주의할 점: .(점)은 현재 디렉토리를 의미하므로, 만약 상위 디렉토리에 변경사항이 있더라도 현재 명령을 실행하는 위치가 하위 폴더 내부라면 상위 폴더의 변경사항은 포함되지 않습니다. 프로젝트 전체의 변경사항을 위치에 상관없이 모두 올리고 싶다면 git add -A 또는 git add --all을 사용하는 것이 더 안전합니다.

3. 커밋까지 완료하려면?

git add .를 수행한 후, 이 변경사항을 최종적으로 저장소에 기록(커밋)하려면 반드시 git commit 명령을 추가로 실행해야 합니다. 보통은 다음과 같이 두

명령을 연속해서 사용합니다.

```
git add .
```

```
git commit -m "커밋 메시지 작성"
```

만약 git add 단계를 생략하고 변경된 파일(수정 및 삭제)을 바로 커밋하고 싶다면 아래와 같이 -a 옵션을 사용할 수도 있습니다. 다만 이 방법은 새롭게 생성된 파일(Untracked)은 인식하지 못하므로 상황에 맞게 주의해서 사용해야 합니다.

```
git commit -am "수정 및 삭제된 파일 바로 커밋"
```